

1. 주제

- Maze Crawler 환경에서 살아남고 경쟁하기 위한 agent 설계 및 구현하기
(벽을 피하고 크리스탈 에너지를 수집)

2. 코드 흐름

- 1) 환경설정: Kaggle-environments 라이브러리 설치 및 게임 환경 만들기
- 2) 유틸리티 함수 정의: 미로의 벽 데이터(비트필드 방식) 해석, 특정 방향의 길 존재 여부 확인, 내 로봇 및 팩토리 식별에 필요한 함수들을 구축하
- 3) Agent_v1 구현: 팩토리 로봇 혼자서 벽이 없으면 북쪽으로 전진하고, 벽을 만나면 점프하거나 동/서쪽으로 우회하는 bug move 방식을 구현
- 4) Agent_2 구현: 1보다 개선된 에이전트로, 팩토리가 '스카우트' 로봇을 생산하고, 이 스카우트가 최근 방문한 경로를 기억하며 맵 내 크리스탈을 수집해 팩토리에 에너지를 배달하는 Snail move 방식을 추가

3. 중요한 코드

- Bug move 우회 로직

```
def factory_bug_north(uid, col, row, jump_cd, obs, config):
```

```
    if has_road(obs, config, col, row, W"NORTHW"): return W"NORTHW"
```

```
    if jump_cd == 0: return W"JUMP_NORTHW" if has_road(obs, config, col, row, W"EASTW"): return W"EASTW"
```

```
    if has_road(obs, config, col, row, W"WESTW"): return W"WESTW" return W"IDLEW"
```

- Snail move 탐색 및 리스트 업데이트

```
for d in (W"NORTHW", W"SOUTHW", W"EASTW", W"WESTW"):
```

```
    if not has_road(obs, config, col, row, d): continue
```

```
    nc, nr = neighbor(col, row, d)
```

```
    if (nc, nr) in state[W"tabuW"]: continue
```

```
    dist = abs(nc - tc) + abs(nr - tr)
```

```
    candidates.append((dist, d))
```

4. 새롭게 알게된 내용/어려운 점/ 배울 점

- 1) 비트필드로 벽 맵핑하기: 미로의 벽을 동서남북(1, 2, 4, 8)라는 비트값의 합으로 표현하고 이를 &로 체크하여 이동가능 여부를 판단하는 인코딩 방식을 썼는데, 이런 방식은 처음 봐서 신기했음
- 2) Env=make("crawl"...) 부분에서 알 수 없는 에러가 났는데, 이 부분을 어떻게 해결하는지 모르겠음
- 3) Baseline: 처음부터 agent_2를 만드는게 아니라, 처음에는 agent_1을 만들고 여기에다가 스카우트 상호작용이 추가된 agent_2를 만들었는데 이 부분을 왜 이렇게 하는지 신기했음. 찾아본 결과 이런 개발 방식이 정석적이라고 함. 앞으로 이렇게 ai를 설계할 때 이런 단계를 참고해서 해야겠다는 생각을 했음.